

Solution Code - Module 3-Core Java - Records

Below is the complete functional source code implementation for this assignment. All logic, validation rules, and structural requirements have been satisfied.

Source File: Q29_Records.java

```
1 | import java.util.Arrays;
2 | import java.util.List;
3 | import java.util.stream.Collectors;
4 |
5 | // Define a record named Person
6 | record Person(String name, int age) {}
7 |
8 | public class Q29_Records {
9 |     public static void main(String[] args) {
10 |         // Create instances and print them
11 |         Person person1 = new Person("Alice", 25);
12 |         Person person2 = new Person("Bob", 17);
13 |         Person person3 = new Person("Charlie", 32);
14 |         Person person4 = new Person("Diana", 15);
15 |         Person person5 = new Person("Evan", 21);
16 |
17 |         System.out.println("=== Person Record Instances ===");
18 |         System.out.println("Person 1: " + person1);
19 |         System.out.println("Person 2: " + person2);
20 |         System.out.println("Name of Person 1: " + person1.name());
21 |         System.out.println("Age of Person 1: " + person1.age());
22 |
23 |         // Use records in a List
24 |         List<Person> people = Arrays.asList(person1, person2, person3, person4, person5);
25 |         System.out.println("\nAll people: " + people);
26 |
27 |         // Filter based on age using Streams (e.g., age >= 18)
28 |         List<Person> adults = people.stream()
29 |             .filter(p -> p.age() >= 18)
30 |             .collect(Collectors.toList());
31 |
32 |         System.out.println("\nAdults (age >= 18):");
33 |         adults.forEach(p -> System.out.println(" - " + p.name() + " (" + p.age() + ")"));
34 |     }
35 | }
```